

C14 - Associations: Aggregation and Composition

CourseAgg, CourseCom, and Copying Objects

Learning Objectives

- Implement a C++ association between Course and Book using a pointer.
- Distinguish aggregation from composition in terms of object ownership and lifetime.
- Implement a copy constructor for a class containing a pointer.
- Implement a copy-assignment operator and understand the difference between copying and assignment.
- Overload the stream insertion operator (`operator<<`).
- Observe object addresses to verify whether Book objects are shared or independently copied.

Background

A *Course* has a *Book*. This relationship can be implemented in two different ways. In aggregation, the Book exists independently of the Course and may be shared by several courses. In composition, the Course owns its Book; each CourseCom has its own Book object, and the Book is destroyed when its CourseCom is destroyed.

The starter classes below intentionally omit several member functions. Your task is to complete those functions while preserving the ownership rules described for each class.

Part 1 — CourseAgg: Aggregation

Complete the following missing functions in **CourseAgg.h**:

- `setBook(Book*)`: store the supplied Book pointer. Do not create a copy of the Book.
- Copy constructor: perform the appropriate copy for aggregation.
- Copy-assignment operator: perform the appropriate copy for aggregation.
- `operator<<`: display the CourseAgg object by using `toString()`.

Important: CourseAgg does not own the Book. Its destructor must not delete `pbook`. The Book should remain available to other CourseAgg objects.

Part 2 — CourseCom: Composition

Complete the following missing functions in **CourseCom.h**:

- `setBook(Book*)`: create an independent Book copy from the supplied Book information.
- Copy constructor: create a new, independent Book containing the same information as the source.
- Copy-assignment operator: replace the current Book with an independent copy of the source Book.
- `operator<<`: display the CourseCom object by using `toString()`.

Important: CourseCom owns its Book. Its destructor must delete the Book that it owns. Do not delete the original Book supplied to the constructor or `setBook()`.

Part 3 — experiment01()

Use the supplied `experiment01()` function to test both classes. For `CourseAgg`, verify that `c1`, `c2`, and `c3` refer to the same `Book` address. For `CourseCom`, verify that `c1`, `c2`, `c3`, and `c4` refer to different `Book` addresses.

Note: `CourseAgg c3 = c1` invokes the copy constructor because `c3` is being initialized. The statement `c4 = c1`, when `c4` already exists, invokes the copy-assignment operator.

Quality Control

- Do not change the data members of the classes.
- Use `Book*` as provided; do not replace it with a different representation.
- `CourseAgg` must not delete the `Book`.
- `CourseCom` must delete the `Book` it owns.
- `CourseAgg` copying should preserve the shared `Book` pointer.
- `CourseCom` copying should create independent `Book` objects.
- The copy-assignment operator must return `CourseCom&` / `CourseAgg&` as appropriate.
- Protect the assignment operator against self-assignment.
- `operator<<` should use `toString()` rather than duplicate its formatting logic.
- The program should compile without memory leaks from the objects created by the experiment.
- No memory leaks.

Self-Assessment

1. In `CourseAgg`, should the copy constructor copy the `Book` object or the `Book` pointer? Explain why.
2. After `a1` is copied to `a2`, what should `a1.getBook()` and `a2.getBook()` return? Should the addresses be equal?
3. Why must `CourseAgg`'s destructor not execute `delete pbook`?
4. In `CourseCom`, why must the copy constructor create a new `Book`?
5. What is the difference between `CourseCom c3 = c1`; and `c4 = c1` when `c4` already exists?
6. Why does `CourseCom`'s destructor need to execute `delete pbook`?
7. Who is responsible for deleting `b1` in `experiment01()`?

Output Sample

Your output should be similar to the following (memory locations will be different)

experiment01 - aggregation and composition

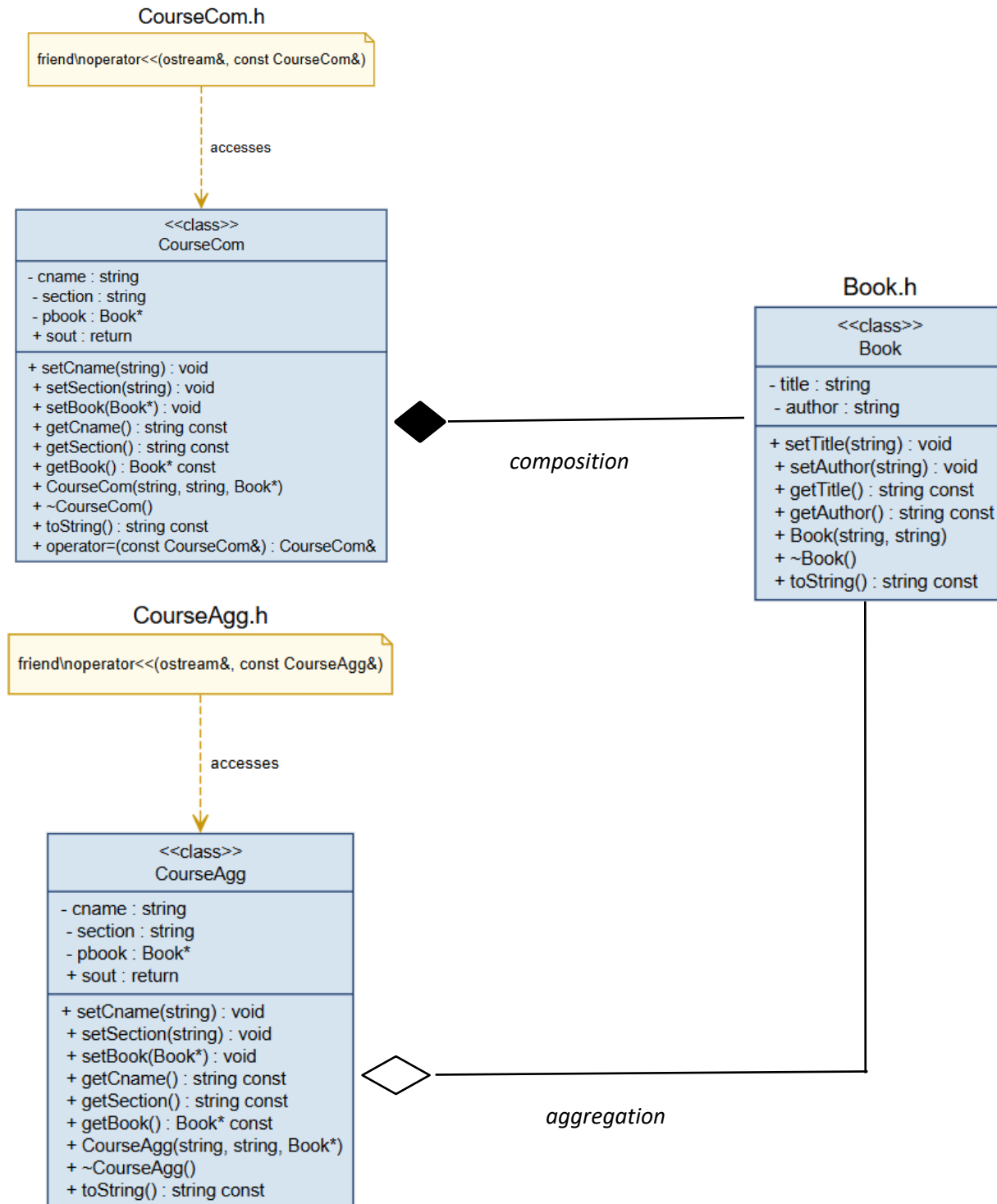
```

--- CourseAgg: Aggregation ---
b1 000002905358C3F0 Book [Title: C++ Programming, Author: T. Gaddis]
a1 00000093F9EFEF20 CourseAgg [ Cname: CS2, Section: S0139, pbook (address): 000002905358C3F0,
    Book info: 000002905358C3F0 Book [Title: C++ Programming, Author: T. Gaddis] ]
a2 00000093F9EFEFA0 CourseAgg [ Cname: CS2, Section: S0139, pbook (address): 000002905358C3F0,
    Book info: 000002905358C3F0 Book [Title: C++ Programming, Author: T. Gaddis] ]
a3 00000093F9EFF020 CourseAgg [ Cname: CS2, Section: S0139, pbook (address): 000002905358C3F0,
    Book info: 000002905358C3F0 Book [Title: C++ Programming, Author: T. Gaddis] ]

--- CourseCom: Composition ---
c1 00000093F9EFF0A0 CourseCom [ Cname: CS2, Section: S0139, pbook (address): 000002905358C6C0,
    Book info: 000002905358C6C0 Book [Title: C++ Programming, Author: T. Gaddis] ]
c2 00000093F9EFF120 CourseCom [ Cname: CS2, Section: S0139, pbook (address): 000002905358C120,
    Book info: 000002905358C120 Book [Title: C++ Programming, Author: T. Gaddis] ]
c3 00000093F9EFF1A0 CourseCom [ Cname: CS2, Section: S0139, pbook (address): 0000029053589C00,
    Book info: 0000029053589C00 Book [Title: C++ Programming, Author: T. Gaddis] ]
c4 before assignment 00000093F9EFF220 CourseCom [ Cname: n.a., Section: n.a., pbook (address): 0000000000000000,
Book: none ]
c4 after assignment 00000093F9EFF220 CourseCom [ Cname: CS2, Section: S0139, pbook (address): 0000029053589CF0,
    Book info: 0000029053589CF0 Book [Title: C++ Programming, Author: T. Gaddis] ]

+ b1 book deleted (Debugging)
+ Book 000002905358C3F0 deteted
+ CourseCom 00000093F9EFF220 deleted, + Book 0000029053589CF0 deleted
+ Book 0000029053589CF0 deteted
+ CourseCom 00000093F9EFF1A0 deleted, + Book 0000029053589C00 deleted
+ Book 0000029053589C00 deteted
+ CourseCom 00000093F9EFF120 deleted, + Book 000002905358C120 deleted
+ Book 000002905358C120 deteted
+ CourseCom 00000093F9EFF0A0 deleted, + Book 000002905358C6C0 deleted
+ Book 000002905358C6C0 deteted
+ CourseAgg 00000093F9EFF020 deleted
+ CourseAgg 00000093F9EFEFA0 deleted
+ CourseAgg 00000093F9EFEF20 deleted
All done!

```



Starter Code — Book.h

```
#pragma once
#include <iostream>
#include <sstream>
using namespace std;

class Book
{
private:
    string title;
    string author;

public:
    void setTitle(string titleValue) { title = titleValue; }
    void setAuthor(string authorValue) { author = authorValue; }

    string getTitle() const { return title; }
    string getAuthor() const { return author; }

    Book(string titleValue = "none", string authorValue = "n.a.")
    {
        setTitle(titleValue);
        setAuthor(authorValue);
    }

    string toString() const
    {
        stringstream sout;
        sout << this << " Book [Title: " << title
            << ", Author: " << author << "];"
        return sout.str();
    }
};
```

Starter Code — CourseAgg.h

```
#pragma once
#include "Book.h"
using namespace std;

//=====
// CourseAgg demonstrates AGGREGATION (CourseAgg <--> Book)
//
// A CourseAgg uses a Book but does NOT own the Book.
// The Book can be shared by several CourseAgg objects.
//
// Key idea:
//   CourseAgg copies the Book POINTER.
//   CourseAgg does NOT create or delete the Book.
//=====

class CourseAgg
{
private:
    string cname;
    string section;
    Book* pbook = nullptr;

public:

    void setCname(string cnameValue)
    {
        cname = cnameValue;
    }

    void setSection(string sectionValue)
    {
```

```

        section = sectionValue;
    }

    // TODO: Implement setBook()
    //
    // Aggregation:
    // Store the supplied pointer.
    // Do NOT create a new Book.
    // Do NOT delete the supplied Book.

    string getName() const
    {
        return cname;
    }

    string getSection() const
    {
        return section;
    }

    Book* getBook() const
    {
        return pbook;
    }

    CourseAgg(string cnameValue = "n.a.",
              string sectionValue = "n.a.",
              Book* pbookValue = nullptr)
    {
        setName(cnameValue);
        setSection(sectionValue);
        setBook(pbookValue);
    }

    ~CourseAgg()
    {
        // CourseAgg does NOT own the Book.
        // Therefore, do NOT delete pbook.

        cout << "+ CourseAgg " << this
              << " deleted" << endl;
    }

    // TODO: Implement the copy constructor.
    //
    // Aggregation:
    // Copy the pointer so that the new CourseAgg
    // refers to the same Book.

    // TODO: Implement the copy-assignment operator.
    //
    // Aggregation:
    // Copy the pointer so that both CourseAgg objects
    // refer to the same Book.

    // TODO: Implement operator<<

    string toString() const
    {
        stringstream sout;

        sout << this << " CourseAgg [ Cname: " << cname
              << ", Section: " << section
              << ", pbook (address): " << pbook;

        if (pbook == nullptr)
        {
            sout << ", Book: none ]";
        }
        else

```

```

        {
            sout << ",\n\t\t\t Book info: "
                  << pbook->toString() << " ]";
        }

        return sout.str();
    }
};

```

Starter Code — CourseCom.h

```

#pragma once
#include "Book.h"
using namespace std;

//=====
// CourseCom demonstrates COMPOSITION (CourseCom <--> Book)
//
// A CourseCom OWNS its Book.
//
// Key idea:
//   CourseCom creates its own Book copy.
//   Each CourseCom has an independent Book object.
//   CourseCom is responsible for deleting its Book.
//=====

class CourseCom
{
private:
    string cname;
    string section;
    Book* pbook = nullptr;

public:

    void setName(string cnameValue)
    {
        cname = cnameValue;
    }

    void setSection(string sectionValue)
    {
        section = sectionValue;
    }

    // TODO: Implement setBook()
    //
    // Composition:
    // If pbookValue is not nullptr:
    //   create a NEW Book containing the same information.
    //
    // If pbookValue is nullptr:
    //   pbook should become nullptr.
    //
    // Remember:
    // CourseCom OWNS the Book.

    string getName() const
    {
        return cname;
    }

    string getSection() const
    {
        return section;
    }
}

```

```

Book* getBook() const
{
    return pbook;
}

CourseCom(string cnameValue = "n.a.",
          string sectionValue = "n.a.",
          Book* pbookValue = nullptr)
{
    cname = cnameValue;
    section = sectionValue;

    // Create an independent Book copy
    // from the supplied Book.

    if (pbookValue != nullptr)
    {
        pbook = new Book(
            pbookValue->getTitle(),
            pbookValue->getAuthor()
        );
    }
}

~CourseCom()
{
    // CourseCom OWNS the Book.
    // TODO: Delete the Book owned by this CourseCom.

    cout << "+ CourseCom " << this
         << " deleted, + Book " << pbook
         << " deleted" << endl;
}

// TODO: Implement the copy constructor.
//
// Composition:
// Create a NEW independent Book containing
// the same information as other.pbook.

// TODO: Implement the copy-assignment operator.
//
// Composition:
// 1. Protect against self-assignment.
// 2. Delete the current Book.
// 3. Create a NEW Book containing the information
//    from other.pbook.

// TODO: Implement operator<<

string toString() const
{
    stringstream sout;

    sout << this << " CourseCom [ Cname: " << cname
         << ", Section: " << section
         << ", pbook (address): " << pbook;

    if (pbook == nullptr)
    {
        sout << ", Book: none ]";
    }
    else
    {
        sout << ",\n\t\t\t\t Book info: "
             << pbook->toString() << " ]";
    }

    return sout.str();
}
};

```

Starter Code — experiment01()

```

void experiment01()
{
    cout << "experiment01 - aggregation and composition" << endl;

    //=====
    // AGGREGATION
    //=====
    cout << "\n--- CourseAgg: Aggregation ---" << endl;

    Book* b1 = new Book("C++ Programming", "T. Gaddis");
    cout << "b1 " << b1->toString() << endl;

    CourseAgg a1("CS2", "S0139", b1);
    cout << "a1 " << a1.toString() << endl;

    // TODO: Test CourseAgg copy constructor.
    CourseAgg a2(a1);
    cout << "a2 " << a2.toString() << endl;

    // TODO: Test CourseAgg copy constructor again.
    CourseAgg a3 = a1;
    cout << "a3 " << a3.toString() << endl;

    // TODO: Display all Book addresses.
    // They should all be the same for aggregation.

    //=====
    // COMPOSITION
    //=====
    cout << "\n--- CourseCom: Composition ---" << endl;

    CourseCom c1("CS2", "S0139", b1);
    cout << "c1 " << c1.toString() << endl;

    // TODO: Test CourseCom copy constructor.
    CourseCom c2(c1);
    cout << "c2 " << c2.toString() << endl;

    // TODO: Test CourseCom copy constructor again.
    CourseCom c3 = c1;
    cout << "c3 " << c3.toString() << endl;

    // TODO: Test CourseCom copy assignment.
    CourseCom c4;
    cout << "c4 before assignment " << c4.toString() << endl;
    c4 = c1;
    cout << "c4 after assignment " << c4.toString() << endl;

    // TODO: Display Book addresses.
    // c1, c2, c3, and c4 should have different Book addresses.

    // b1 is NOT owned by CourseAgg or CourseCom.
    // It was created here, so it must be deleted here.
    delete b1;
}

```

Partial Solution – CourseCom.h

```
#pragma once
#include "Book.h"
using namespace std;

//The class CourseCom uses COMPOSITION (CourseCom <--> Book)

class CourseCom
{
private:
    string cname;
    string section;
    Book* pbook = nullptr;

public:
    void setName(string cnameValue) { cname = cnameValue; }
    void setSection(string sectionValue) { section = sectionValue; }

    void setBook(Book* pbookValue) {
        // Prevent deleting the Book before copying it
        if (pbook == pbookValue) return;
        // Delete the current Book owned by this CourseCom
        if (pbook != nullptr) { delete pbook; }

        if (pbookValue != nullptr)
            // Create an independent copy
            pbook = new Book(
                pbookValue->getTitle(),
                pbookValue->getAuthor()
            );
        else
            //the course will not have a textbook
            pbook = nullptr;
    }

    string getName() const { return cname; }
    string getSection() const { return section; }
    Book* getBook() const { return pbook; }

    CourseCom(string cnameValue = "n.a.",
              string sectionValue = "n.a.",
              Book* pbookValue = nullptr)
    {
        setName(cnameValue);
        setSection(sectionValue);
        setBook(pbookValue); // Create a distinct Book copy
                             // using the provided pointer
    }

    ~CourseCom()
    {
        // Destroy both the Course and its associated Book copy
        cout << "+ CourseCom " << this << " deleted"
              << ", + Book " << pbook << " deleted" << endl;
        delete pbook;
    }

    string toString() const {
```

```

        stringstream sout;
        //Show course data
        sout << this << " CourseCom [ Cname: " << cname
            << ", Section: " << section
            << ", pbook (address): " << pbook;
        //Show book information
        if (pbook == nullptr) {
            sout << ", Book: none ]";
        }
        else {
            sout << ",\n\t\t\t\t Book info: " << pbook->toString() << " ]";
        }
        return sout.str();
    }

    //=====
    //TODO - copy constructor needed!
    //TODO - copy assignment (operator=) needed
    //TODO - override operator<<
    //=====
    // DONE!
    friend ostream& operator<< (ostream& sout, const CourseCom& c) {
        sout << c.toString();
        return sout;
    }

    CourseCom(const CourseCom& other) {
        setName(other.cname);
        setSection(other.section);
        setBook(other.pbook);
    }

    CourseCom& operator=(const CourseCom& other) {
        if (this != &other) {
            setName(other.cname);
            setSection(other.section);
            setBook(other.getBook());
        }
        return *this;
    }
};

```